JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AbstractExecutorService

java.lang.Object
    java.util.concurrent.AbstractExecutorService

All Implemented Interfaces:

AutoCloseable, Executor, ExecutorService

Direct Known Subclasses:

ForkJoinPool, ThreadPoolExecutor

```
public abstract class AbstractExecutorService
extends Object
implements ExecutorService
```

Provides default implementations of ExecutorService methods other than Executor.execute(Runnable). This class implements the submit, invokeAny and invokeAll methods using a RunnableFuture returned by newTaskFor, which defaults to the FutureTask class provided in this package. For example, the implementation of submit(Runnable) creates an associated RunnableFuture that is executed and returned. Subclasses may override the newTaskFor methods to return RunnableFuture implementations other than FutureTask.

Extension example. Here is a sketch of a class that customizes ThreadPoolExecutor to use a CustomTask class instead of the default FutureTask:

```
public class CustomThreadPoolExecutor extends ThreadPoolExecutor {

  static class CustomTask<V> implements RunnableFuture<V> { ... }

  protected <V> RunnableFuture<V> newTaskFor(Callable<V> c) {
      return new CustomTask<V>(c);
  }
  protected <V> RunnableFuture<V> newTaskFor(Runnable r, V v) {
      return new CustomTask<V>(r, v);
  }
  // ... add constructors, etc.
}
```

Since:

1.5

Constructor Summary

Constructors

Constructor	Description
`AbstractExecutorService()`	Constructor for subclasses to call.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`<T> List<Future<T>>`	`invokeAll(Collection<? extends Callable<T>> tasks)`	Executes the given tasks, returning a list of Futures holding their status and results when all complete.
`<T> List<Future<T>>`	`invokeAll(Collection<? extends Callable<T>> tasks, long timeout, TimeUnit unit)`	Executes the given tasks, returning a list of Futures holding their status and results when all complete or the timeout expires, whichever happens first.
`<T> T`	`invokeAny(Collection<? extends Callable<T>> tasks)`	Executes the given tasks, returning the result of one that has completed successfully (i.e., without throwing an exception), if any do.
`<T> T`	`invokeAny(Collection<? extends Callable<T>> tasks, long timeout, TimeUnit unit)`	Executes the given tasks, returning the result of one that has completed successfully (i.e., without throwing an exception), if any do before the given timeout elapses.

protected <T> RunnableFuture <T>	**newTaskFor** (Runnable runnable, T value)	Returns a RunnableFuture for the given runnable and default value.
protected <T> RunnableFuture <T>	**newTaskFor** (Callable<T> callable)	Returns a RunnableFuture for the given callable task.
Future<?>	**submit**(Runnable task)	Submits a Runnable task for execution and returns a Future representing that task.
<T> Future<T>	**submit**(Runnable task, T result)	Submits a Runnable task for execution and returns a Future representing that task.
<T> Future<T>	**submit** (Callable<T> task)	Submits a value-returning task for execution and returns a Future representing the pending results of the task.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Methods declared in interface Executor

execute

Methods declared in interface ExecutorService

awaitTermination, close, isShutdown, isTerminated, shutdown, shutdownNow

Constructor Details

AbstractExecutorService

```
public AbstractExecutorService()
```

Constructor for subclasses to call.

Method Details

newTaskFor

```
protected <T> RunnableFuture<T> newTaskFor(Runnable runnable,
                                           T value)
```

Returns a `RunnableFuture` for the given runnable and default value.

Type Parameters:

`T` - the type of the given value

Parameters:

`runnable` - the runnable task being wrapped

`value` - the default value for the returned future

Returns:

a `RunnableFuture` which, when run, will run the underlying runnable and which, as a `Future`, will yield the given value as its result and provide for cancellation of the underlying task

Since:

1.6

newTaskFor

```
protected <T> RunnableFuture<T> newTaskFor(Callable<T> callable)
```

Returns a `RunnableFuture` for the given callable task.

Type Parameters:

`T` - the type of the callable's result

Parameters:

`callable` - the callable task being wrapped

Returns:

a `RunnableFuture` which, when run, will call the underlying callable and which, as a `Future`, will yield the callable's result as its result and provide for cancellation of the underlying task

Since:

1.6

submit

```
public Future<?> submit(Runnable task)
```

Description copied from interface: ExecutorService

Submits a Runnable task for execution and returns a Future representing that task. The Future's `get` method will return `null` upon *successful* completion.

Specified by:

`submit` in interface `ExecutorService`

Parameters:

`task` - the task to submit

Returns:

a Future representing pending completion of the task

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if the task is null

submit

```
public <T> Future<T> submit(Runnable task,
                            T result)
```

Description copied from interface: ExecutorService

Submits a Runnable task for execution and returns a Future representing that task. The Future's `get` method will return the given result upon successful completion.

Specified by:

`submit` in interface `ExecutorService`

Type Parameters:

`T` - the type of the result

Parameters:

`task` - the task to submit

`result` - the result to return

Returns:

a Future representing pending completion of the task

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if the task is null

submit

```
public <T> Future<T> submit(Callable<T> task)
```

Description copied from interface: ExecutorService

Submits a value-returning task for execution and returns a Future representing the pending results of the task. The Future's `get` method will return the task's result upon successful completion.

If you would like to immediately block waiting for a task, you can use constructions of the form `result = exec.submit(aCallable).get();`

Note: The `Executors` class includes a set of methods that can convert some other common closure-like objects, for example, `PrivilegedAction` to `Callable` form so they can be submitted.

Specified by:

`submit` in interface `ExecutorService`

Type Parameters:

T - the type of the task's result

Parameters:

`task` - the task to submit

Returns:

a Future representing pending completion of the task

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if the task is null

invokeAny

```
public <T> T invokeAny(Collection<? extends Callable<T>> tasks)
                throws InterruptedException,
                       ExecutionException
```

Description copied from interface: ExecutorService

Executes the given tasks, returning the result of one that has completed successfully (i.e., without throwing an exception), if any do. Upon normal or exceptional return, tasks that have not completed are cancelled. The results of this method are undefined if the given collection is modified while this operation is in progress.

Specified by:

`invokeAny` in interface `ExecutorService`

Type Parameters:

`T` - the type of the values returned from the tasks

Parameters:

`tasks` - the collection of tasks

Returns:

the result returned by one of the tasks

Throws:

`InterruptedException` - if interrupted while waiting

`NullPointerException` - if tasks or any element task subject to execution is `null`

`IllegalArgumentException` - if tasks is empty

`ExecutionException` - if no task successfully completes

`RejectedExecutionException` - if tasks cannot be scheduled for execution

invokeAny

```
public <T> T invokeAny(Collection<? extends Callable<T>> tasks,
                       long timeout,
                       TimeUnit unit)
                throws InterruptedException,
                       ExecutionException,
                       TimeoutException
```

Description copied from interface: `ExecutorService`

Executes the given tasks, returning the result of one that has completed successfully (i.e., without throwing an exception), if any do before the given timeout elapses. Upon normal or exceptional return, tasks that have not completed are cancelled. The results of this method are undefined if the given collection is modified while this operation is in progress.

Specified by:

`invokeAny` in interface `ExecutorService`

Type Parameters:

`T` - the type of the values returned from the tasks

Parameters:

`tasks` - the collection of tasks

`timeout` - the maximum time to wait

`unit` - the time unit of the timeout argument

Returns:

the result returned by one of the tasks

Throws:

`InterruptedException` - if interrupted while waiting

`NullPointerException` - if tasks, or unit, or any element task subject to execution is `null`

`TimeoutException` - if the given timeout elapses before any task successfully completes

`ExecutionException` - if no task successfully completes

`RejectedExecutionException` - if tasks cannot be scheduled for execution

invokeAll

```
public <T> List<Future<T>> invokeAll
(Collection<? extends Callable<T>> tasks)
                              throws InterruptedException
```

Description copied from interface: `ExecutorService`

Executes the given tasks, returning a list of Futures holding their status and results when all complete. `Future.isDone()` is `true` for each element of the returned list. Note that a *completed* task could have terminated either normally or by throwing an exception. The results of this method are undefined if the given collection is modified while this operation is in progress.

Specified by:

`invokeAll` in interface `ExecutorService`

Type Parameters:

`T` - the type of the values returned from the tasks

Parameters:

`tasks` - the collection of tasks

Returns:

a list of Futures representing the tasks, in the same sequential order as produced by the iterator for the given task list, each of which has completed

Throws:

`InterruptedException` - if interrupted while waiting, in which case unfinished tasks are cancelled

`NullPointerException` - if tasks or any of its elements are `null`

`RejectedExecutionException` - if any task cannot be scheduled for execution

invokeAll

```
public <T> List<Future<T>> invokeAll
(Collection<? extends Callable<T>> tasks,
 long timeout,
 TimeUnit unit)
                              throws InterruptedException
```

Description copied from interface: `ExecutorService`

Executes the given tasks, returning a list of Futures holding their status and results when all complete or the timeout expires, whichever happens first. `Future.isDone()` is `true` for each element of the returned list. Upon return, tasks that have not completed are cancelled. Note that a *completed* task could have terminated either normally or by throwing an exception. The results of this method are undefined if the given collection is modified while this operation is in progress.

Specified by:

`invokeAll` in interface `ExecutorService`

Type Parameters:

T - the type of the values returned from the tasks

Parameters:

`tasks` - the collection of tasks

`timeout` - the maximum time to wait

`unit` - the time unit of the timeout argument

Returns:

a list of Futures representing the tasks, in the same sequential order as produced by the iterator for the given task list. If the operation did not time out, each task will have completed. If it did time out, some of these tasks will not have completed.

Throws:

`InterruptedException` - if interrupted while waiting, in which case unfinished tasks are cancelled

`NullPointerException` - if tasks, any of its elements, or unit are `null`

`RejectedExecutionException` - if any task cannot be scheduled for execution

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other

countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.